

Sistemas Distribuidos y Paralelos

Ingeniería en Computación



Modelo de programación sobre memoria distribuida

Universidad Nacional de La Plata



Facultad de Informática





1

Modelo de memoria distribuida

2

MPI

Funcionamiento, compilación y ejecución. Estructura de programa. Comunicación: Parámetros comunes de las funciones MPI, Comunicación punto a punto, Comunicación Colectiva. Afinidad. Ocultamiento de la latencia. Comentarios finales sobre MPI.



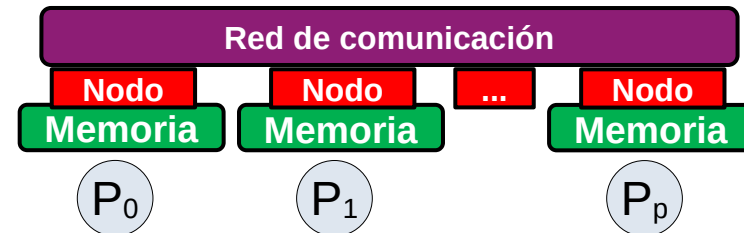
1

Modelo de memoria distribuida

Modelo de memoria distribuida

4

- **Procesos** independientes, cada uno con su **propio espacio de direcciones**, que se comunican y se sincronizan mediante el intercambio de **mensajes**.
 - No existe memoria compartida entre ellos.
 - Un proceso no puede acceder al espacio de memoria de otro proceso.
 - **No hilos** porque comparten memoria de un mismo proceso y no representan unidades de memoria independiente.
- Surge a partir de los primeros **clusters moncore**.
- Herramientas para el desarrollo de aplicaciones utilizando este modelo:
 - **PVM - 1989**: Impulsó el uso de clusters. **Hoy en desuso**.
 - **MPI**: Estándar vigente. C/C++ - Fortran.
 - Otras: Sockets, RMI, etc.

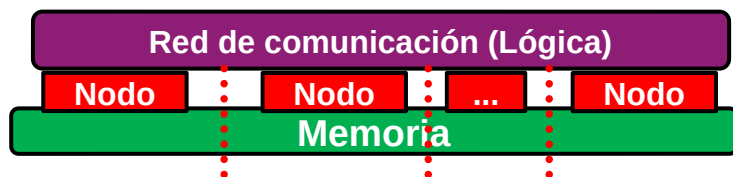


Modelo de memoria distribuida

Modelos de programación paralela – Modelos de arquitectura paralela

5

- Puede utilizarse en arquitecturas de memoria compartida, arquitecturas de memoria distribuida o híbridos.



		Software		
		Memoria Compartida (Ej: OpenMP)	Memoria Distribuida (Ej: MPI)	Híbrido (EJ: MPI + OpenMP)
Hardware	Memoria Compartida (Ej: multicore)	Trivial (Ej: OpenMP sobre Multicore)	Posible (Ej: MPI sobre multicore)	Posible (extraño) (Ej: MPI + OpenMP sobre multicore)
	Memoria Distribuida (Ej: cluster moncore ¹)	(NO ADECUADO) Single System Image(SSI) Overhead	Trivial (Ej: MPI sobre cluster)	Posible (muy extraño) (Ej: MPI + OpenMP sobre cluster moncore)
	Híbrido (Ej: cluster multicore)	(NO ADECUADO) Single System Image(SSI) Overhead	Posible (Ej: MPI sobre cluster multicore)	Trivial (Ej: MPI + OpenMP sobre cluster multicore)

¹Actualmente, los clusters moncore cayeron en desuso pero se mencionan para ejemplificar.

MPI

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales obre MPI

- **Estándar¹ Message Passing Interface (MPI)**: define sintaxis y semántica de funciones para pasaje de mensajes. **No define los comandos.**
- Varias distribuciones que implementan el estándar en C/C++ y Fortran:
 - OpenMPI
 - Mpich
 - Otros...
 - *Para otros lenguajes (**Java o Python**) son **wrappers** bajo lenguaje **C**.*
- Nace en **1994** como iniciativa de un consorcio, **MPI Forum**:
 - IBM, Intel, Cray Research, Argonne National Laboratory, Universidad de Illinois...
- **SPMD (Single Program, Multiple Data)**: todos los procesos ejecutan el mismo código.
- **Runtime system o Runtime environment (ORTE)** gestiona el entorno de ejecución.

¹<http://www.mpi-forum.org>

MPI

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

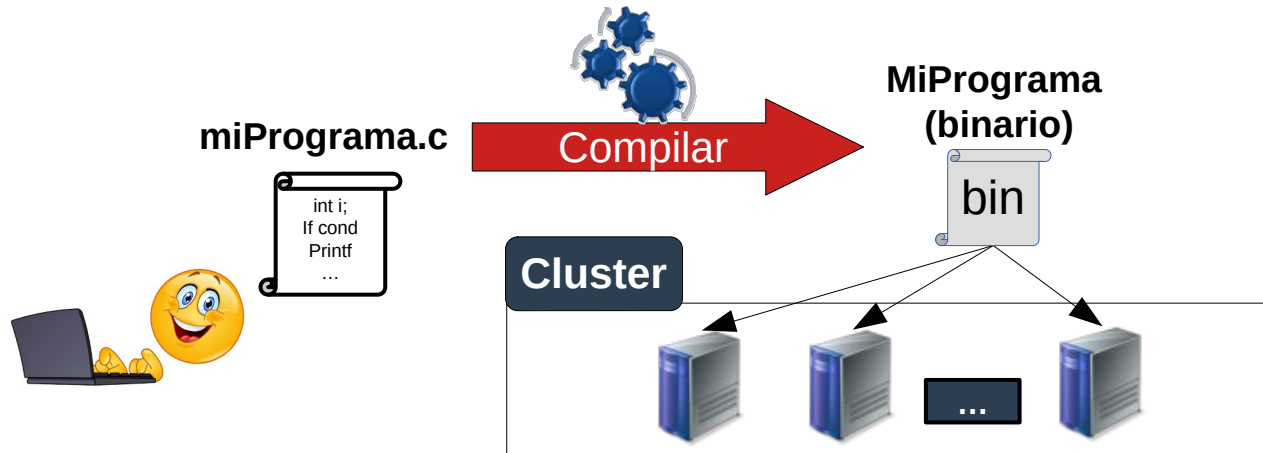
Comentarios finales obre MPI

MPI

Funcionamiento

9

- Suponemos:
 - Un **cluster** con varios nodos conectados por una red física.
 - Distribución **OpenMPI**.
- Cada nodo debe conocer el **archivo ejecutable (binario)**.
- El programador debe compilar el código fuente y distribuir el binario.
 - MPI no lo hace



MPI

Compilación

10

- Compilación MPI:

```
mpicc -o miPrograma miPrograma.c
```

- La compilación depende de las características del cluster:

Cluster Homogéneo

- Nodos iguales.
- Compilar el código fuente en un nodo.
- Distribuir el binario o compartirlo mediante un sistema de archivos distribuido (**NFS**).

miPrograma.c

```
int i;  
If cond  
Printf  
...
```

Compilar

MiPrograma



Cluster Heterogéneo

- Nodos diferentes.
- Compilar el código fuente en cada nodo. Cada nodo tendrá su archivo binario.

miPrograma.c

```
int i;  
If cond  
Printf  
...
```



Compilar

Compilar

Compilar

MiPrograma

MiPrograma

MiPrograma



MPI

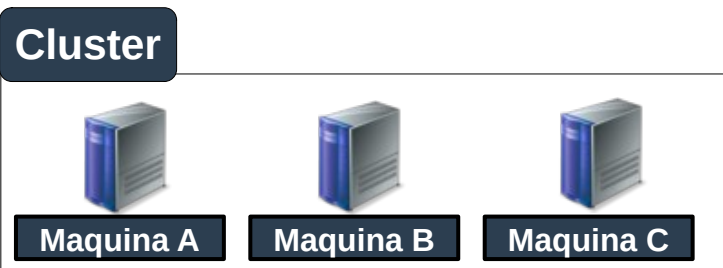
Ejecución

11

- Ejecución MPI:

```
mpirun -np NrProcesos -machinefile maquinas miPrograma
```

- **NrProcesos:** número total de procesos a crear.
- **maquinas:** archivo que contiene el nombre de los nodos.
 - Varía dependiendo de la distribución.



Archivo “maquinas”

maquinaA slots=2
maquinaB slots=1
maquinaC slots=1

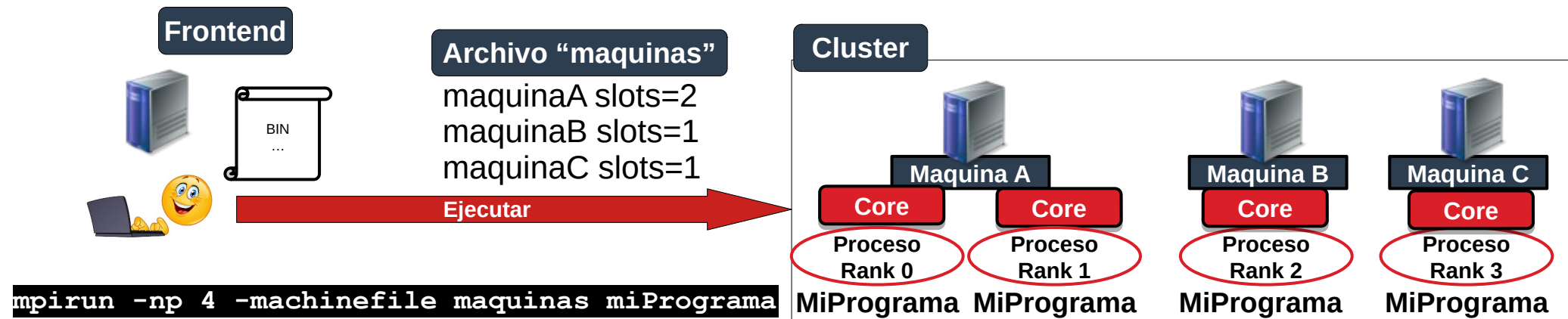
slots: número de procesos que se le enviarán a cada nodo

MPI

Ejecución

12

- Habitualmente, se ejecuta desde un **frontend** (nodo de gestión y servicios)
El **Runtime System** ejecuta una copia del programa por unidad de procesamiento:
 - **Cada unidad de procesamiento ejecuta una copia del mismo programa**
- Cada proceso recibe un identificador único (**rank**).
 - Generalmente, en el orden del archivo de nodos.



Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales sobre MPI

MPI

Estructura de programa

14

```
#include<mpi.h>
```

```
void fA() { // Comportamiento A }
```

```
void fB() { // Comportamiento B }
```

```
void fN() { //Comportamiento N }
```

```
...
```

```
int main(int argc, char** argv){
```

```
int id;
```

```
int nProcs;
```

```
MPI_Init(&argc, &argv);
```

```
MPI_Comm_rank(MPI_COMM_WORLD, &id);
```

```
MPI_Comm_size(MPI_COMM_WORLD, &nProcs);
```

```
...
```

```
if(id == 0) fA();
```

```
else if (id == 1) fB();
```

```
else if (...)
```

```
...
```

```
else fN();
```

```
MPI_Finalize();
```

```
return(0);
```

```
}
```

Encabezado MPI

Toda función MPI lleva **prefijo MPI_** y retorna **MPI_SUCCESS** si tuvo éxito o un **código de error**

Cuatro funciones principales:

MPI_Init: inicializa el entorno.
Recibe los parámetros del programa C.
Primera función que debe ejecutarse.

MPI_Comm_rank: Para obtener el rank del proceso
Numerados desde 0.

MPI_Comm_size: Número de procesos creados.

MPI_Finalize: Finaliza el entorno.
Última función que debe ejecutarse
Antes de la sentencia return.

Comunicadores: para topologías virtuales
Siempre **MPI_COMM_WORLD**: todos los procesos

```
tipo_t miVariable;
```

```
int main(int argc, char** argv){  
    int miID;  
    int nrProcs;  
  
    MPI_Init(&argc, &argv);  
    MPI_Comm_rank(MPI_COMM_WORLD, &miID);  
    MPI_Comm_size(MPI_COMM_WORLD, &nrProcs);  
  
    ...  
  
    MPI_Finalize();  
    return(0);  
}
```

Una variable definida en ésta área...

NO ES UNA VARIABLE COMPARTIDA

...entre los procesos.

En un modelo de memoria distribuida:

NO HAY VARIABLES COMPARTIDAS



Recordar:



Cada unidad de procesamiento ejecuta una copia del mismo programa.

Existirá una copia de la variable para cada proceso.

VARIABLE LOCAL Y PRIVADA



Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales obre MPI

- Operaciones básicas de comunicación:
 - Envío (**Send**):
 - **Síncrono**: emisor **espera** recepción del mensaje
 - **Asíncrono**: emisor **no espera** recepción de mensaje y continúa la ejecución
 - Recepción (**Receive**): **Síncrono**, receptor **espera** recibir el mensaje.
- En la práctica, debe considerarse la **semántica del send asíncrono**:



```
Process Emisor{  
  mensaje_t m[];  
  m=data();  
  send(receptor,m);  
  m=data();  
}
```

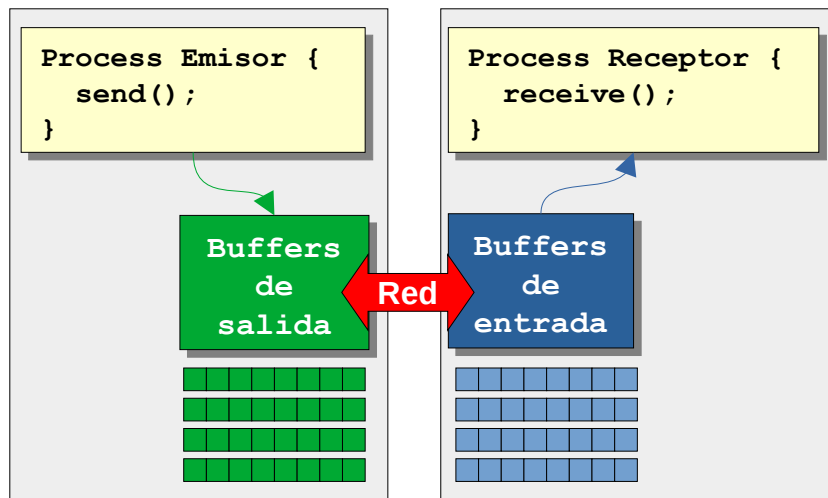


```
Process Receptor{  
  mensaje_t m[];  
  ...  
  receive(emisor, m);  
  ...  
}
```

El emisor continua y modifica la variable del mensaje
¿Y si el mensaje anterior demora en salir?



- Entender el modelo de comunicación real:



Importa el mensaje no el receptor



Clasificación de operaciones:

Coordinación emisor-receptor:

- Síncronas:** demoran al emisor hasta que el receptor comienza la recepción.
- Asíncronas:** si el emisor requiere que el receptor reciba o no el mensaje.

Control de programa:

- Bloqueantes:** (*envío o recepción*) no retorna el control al programa hasta completar la comunicación.
(*send = mensaje en buffer seguro*)
- No Bloqueantes:** retorna el control de *inmediato*.

¿Asíncronas? ¿No bloqueantes?

- **Bloqueante – Síncrona:** El emisor espera que el receptor inicie la recepción.
- **Bloqueante – Asíncrona:** El emisor espera que el mensaje se encuentre en un buffer seguro, no espera al receptor.
- **No Bloqueante – Síncrona:** La operación (envío o recepción) sigue en segundo plano pero no se completa hasta que el receptor comience a recibir.
- **No Bloqueante – Asíncrona:** La operación sigue en segundo plano sin necesidad de coordinación con el receptor, puede usarse un **buffer**.

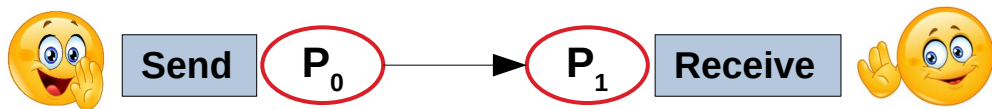


- **Comunicación** entre procesos mediante **pasaje de mensajes**.
- Clasificación de operaciones de comunicación:

Punto a Punto

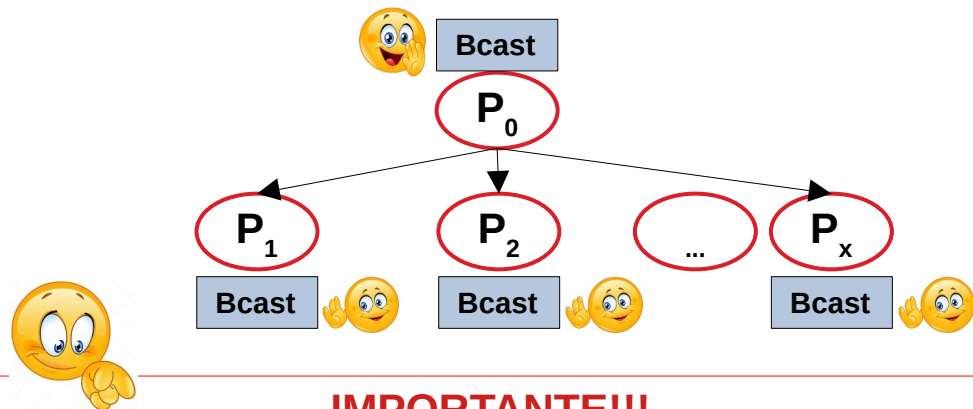
Comunican sólo **dos procesos**.

Un proceso hace un “**tipo**” de **send** y otro proceso hace un “**tipo**” de **receive**.



Colectivas

Comunican **varios procesos**:



IMPORTANTE!!!

Todos los procesos ejecutan la **misma función de comunicación colectiva**.

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales sobre MPI

Parámetros comunes de las funciones MPI

- **Mensaje:** representa el mensaje a enviar o recibir
- **Tipo:** el tipo de datos de los elementos del mensaje:

MPI_BYTE	MPI_SHORT	MPI_INT	MPI_LONG
MPI_FLOAT	MPI_DOUBLE	MPI_UNSIGNED_CHAR	MPI_UNSIGNED_SHORT
MPI_UNSIGNED	MPI_UNSIGNED_LONG	MPI_LONG_DOUBLE	MPI_LONG_LONG_INT

- **Tamaño del mensaje:** cantidad de elementos consecutivos del tipo anterior en el mensaje
- **Comunicador:** el comunicador que agrupa a los procesos
- **Source:** cuando se recibe, identificador de proceso del cual se espera recibir el mensaje
- **Dest:** cuando se envía, identificador del proceso destino del mensaje.
- **Tag:** identificar y distinguir mensajes dentro de una comunicación (símil canales).
- **MPI_ANY_SOURCE** y **MPI_ANY_TAG:** constantes cuando se recibe, desde cualquier origen y por cualquier tag, respectivamente.
 - **MPI_Status:** registro para obtener información del mensaje recibido.

MPI

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales obre MPI

```
int MPI_Send(const void* m,  
             int count,  
             MPI_Datatype dtype,  
             int dest,  
             int tag,  
             MPI_Comm comm)
```

- **Envío bloqueante:** No retorna el control al programa hasta que el mensaje haya sido almacenado de forma segura, de manera que el emisor pueda modificar libremente el mensaje enviado.

m: mensaje.

count: cantidad de elementos del mensaje.

dtype: tipo de datos de los elementos del mensaje.

dest: id del proceso receptor.

tag: tag que identifica el mensaje.

comm: comunicador.

Comunicación punto a punto - MPI_Recv

```
int MPI_Recv(void *buffer,  
             int count,  
             MPI_Datatype dtype,  
             int source,  
             int tag,  
             MPI_Comm comm,  
             MPI_Status *status)
```

- **Recepción bloqueante:** No retorna el control al programa hasta que el mensaje haya sido recibido.

m: mensaje.

count: cantidad de elementos que tiene el mensaje.

dtype: tipo de datos de los elementos del mensaje.

source: id del proceso emisor. **MPI_ANY_SOURCE** recibe de cualquier proceso.

tag: tag que identifica el mensaje. **MPI_ANY_TAG** si recibe por cualquier tag.

comm: comunicador.

status: puntero a la estructura **MPI_Status**. **MPI_STATUS_IGNORE** si no se utiliza.

status.MPI_SOURCE
status.MPI_TAG

Comunicación punto a punto – Ejemplo

26

```
#include <mpi.h>
#define N ...

void emisor() {
    int m[N];
    ... // Inicializar el mensaje
    MPI_Send(m, N, MPI_INT, 1, 7, MPI_COMM_WORLD);
}

void receptor() {
    int m[N];
    MPI_Recv(m, N, MPI_INT, 0, 7, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    ... // Usar el mensaje recibido
}

int main(int argc, char *argv[]) {
    int id;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    if (id == 0) { emisor(); }
    else if (id == 1) { receptor(); }
    MPI_Finalize();
    return 0;
}
```

Alternativa al receptor

```
void receptor() {
    int m[N];
    MPI_Status status;
    int source;
    int tag;
    MPI_Recv(m, N, MPI_INT,
             MPI_ANY_SOURCE,
             MPI_ANY_TAG,
             MPI_COMM_WORLD, &status);
    source = status.MPI_SOURCE;
    tag = status.MPI_TAG;
    ... // Usar el mensaje recibido
}
```



¿Si no sabemos cuantos elementos recibiremos?

```
MPI_Get_count(const MPI_Status *status,  
              MPI_Datatype dtype,  
              int *count)
```



status: puntero a la variable de tipo MPI_Status de la operación de recepción previa.

dtype: tipo de datos de los elementos recibidos.

count: puntero donde MPI guardará la cantidad de datos realmente recibidos.

- La operación de recepción debe tener un valor lo suficientemente alto en el parámetro **count** para que el buffer pueda contener todos los datos sin desbordarse.

```
void receptor() {  
    int m[N];  
    MPI_Status status;  
    MPI_Recv(m, N, MPI_INT, 0, 7, MPI_COMM_WORLD, &status);  
    int real_count;  
    MPI_Get_count(&status, MPI_INT, &real_count);  
    ... // Usar el mensaje recibido  
}
```


Comunicación punto a punto – Variantes o Modos

- Distintas **variantes** o **modos** de comunicación punto a punto indicadas bajo un prefijo:

Prefijo B

Buffered
(Bloqueante)

Prefijo R

Ready
(Bloqueante)

Prefijo S

Synchronous
(Bloqueante)

Prefijo I

Immediate
(NO Bloqueante)
(Asíncrona)



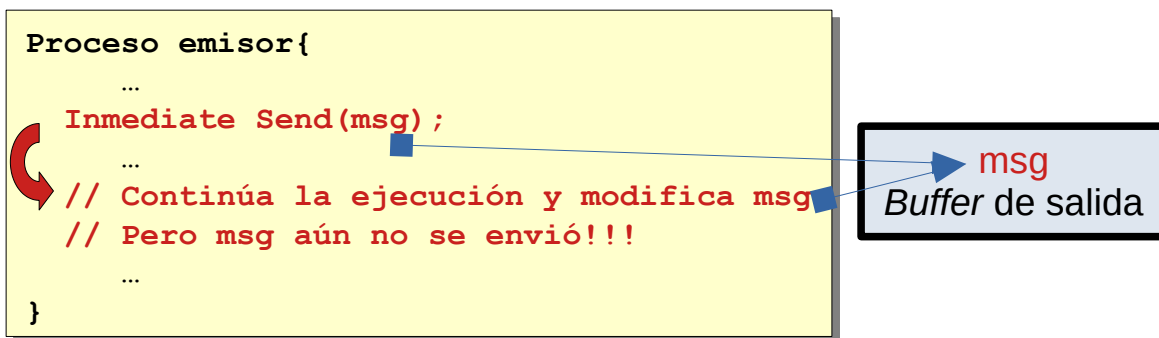
- **MPI_Bsend, MPI_Rsend y MPI_Ssend:** mismo prototipo que **MPI_Send**.
 - Envíos bloqueantes.
- **MPI_Bsend:** *buffer* gestionado por el usuario (**Error por overflow del buffer**).
 - Retorna cuando el mensaje fue copiado al *buffer* de usuario.
- **MPI_Rsend:** envío *ready*. (no requiere buffer) (**Error si el receptor no está listo**)
 - Retorna cuando el receptor está preparado para recibir el mensaje.
- **MPI_Ssend:** síncrono.
 - No retorna hasta que el receptor haya empezado a recibir el mensaje.
 - Garantiza sincronización entre dos procesos y que el mensaje pueda ser reutilizado.

Comunicación punto a punto – Modo Inmediato

- El modo I (*immediate*) es no bloqueante y asíncrono: continúa la ejecución independientemente de lo que ocurra con el mensaje.

IMPORTANTE!!!

Podría modificarse un mensaje que aún no fue enviado !!!



Las operaciones se ejecutan en **segundo plano** por el **Runtime System**.
Parámetro adicional de tipo **MPI_Request** para consultar el progreso de la operación.

- MPI_Isend: *immediate send*.

```
int MPI_Isend(const void* m,
              int count,
              MPI_Datatype dtype,
              int dest,
              int tag,
              MPI_Comm comm,
              MPI_Request *req)
```

- MPI_Irecv: *immediate receive*.



```
int MPI_Irecv(void *buffer,
              int count,
              MPI_Datatype dtype,
              int source,
              int tag,
              MPI_Comm comm,
              MPI_Request *req)
```

MPI_Request reemplaza a MPI_Status

- Dos operaciones relacionadas al **progreso de la operación**:

- **MPI_Wait**: espera que la comunicación finalice.

```
int MPI_Wait(MPI_Request *request, MPI_Status *status)
```

- **MPI_Test**: chequea el progreso de la comunicación.

```
int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)
```

- **flag**: **1** si la comunicación terminó, **0** en caso contrario.

- En ambos casos:

- **request**: el parámetro que se pasó a **MPI_Isend** o **MPI_Irecv**.
- **status**: retorna información de la comunicación.

- El modo "*immediate*" puede combinarse con los otros modos:
 - `MPI_IBsend`
 - `MPI_IRsend`
 - `MPI_ISsend`
- En todos los casos el emisor continua inmediatamente.
 - Utilizan el parámetro `MPI_Request` y se requiere `MPI_Wait` o `MPI_Test` relacionadas al progreso de la operación.
- El **Runtime System** de **MPI** se encarga de gestionar la comunicación.
 - Se aplican las mismas reglas que `MPI_Bsend`, `MPI_Rsend` y `MPI_Ssend` pero en segundo plano.

- Comprobaciones de recepción de un mensaje sin necesidad de recibirlo:
 - **MPI_Probe**: comprobación bloqueante. Espera por la disponibilidad de un mensaje.

```
int MPI_Probe(int source, int tag, MPI_Comm comm, MPI_Status *status)
```
 - **MPI_Iprobe**: comprobación no bloqueante. Devuelve el control al programa inmediatamente.

```
int MPI_Iprobe(int source, int tag, MPI_Comm comm, int *flag, MPI_Status *status)
```

 - **flag**: **1** si hay al menos un mensaje, **0** en caso contrario.
- En ambos casos se utiliza el campo **status** para obtener la información del emisor o la longitud del mensaje recibido.

- Alternativa cuando se desconoce la longitud del mensaje:



- Se utiliza `MPI_Probe` para obtener la longitud del mensaje (campo `status` y la función `MPI_Get_count`).
- Puede alocarse un mensaje de tamaño dinámico en función del valor anterior.

```
void receptor() {  
    MPI_Status status;  
  
    MPI_Probe(MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);  
  
    int count;  
    MPI_Get_count(&status, MPI_INT, &count);  
    int *m = malloc(count * sizeof(int));  
    MPI_Recv(m, count, MPI_INT, status.MPI_SOURCE, status.MPI_TAG, MPI_COMM_WORLD, &status);  
    ... // Usar el mensaje recibido  
  
    free(m);  
}
```

MPI

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales obre MPI

- Las operaciones colectivas involucran mas de dos procesos en la comunicación.
 - Barrera distribuida (MPI_Barrier)
 - Distribución de datos (MPI_Bcast, **SPMD** – MPI_Scatter y MPI_Gather,...)
 - Difícil y propenso a errores con operaciones punto a punto.
 - Son **bloqueantes** (existen los modos l) pero **no** necesariamente **sincrónicas**.
 - La finalización indica que el proceso que invocó a la función es libre de modificar el mensaje. No indica que otros procesos del grupo hayan completado o iniciado la operación (salvo descripción que indique lo contrario).
 - Puede, o no, tener el efecto de sincronizar todos los procesos participantes.

Todos los procesos deben invocar la misma función, aunque cumplan roles diferentes.

Algunos procesos utilizan todos los parámetros, otros solo usan los que corresponden a su rol.

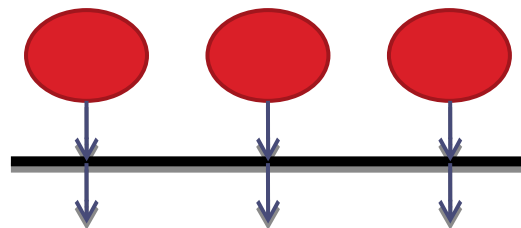
Se deben definir los parámetros que no se usen para cumplir con la sintaxis de las funciones.



- **MPI_Barrier:** barrera distribuida.

```
int MPI_Barrier(MPI_Comm comm)
```

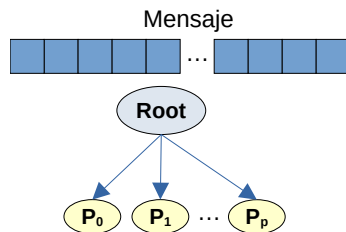
- Demora cada proceso hasta que **MPI_Barrier** sea ejecutada por todos los procesos pertenecientes al comunicador especificado (**comm**).



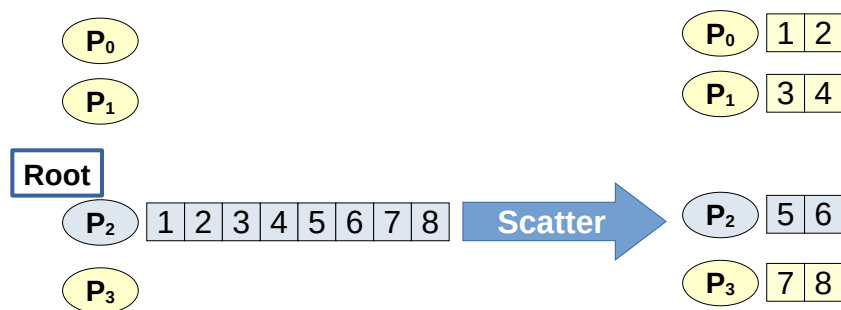
- **MPI_Bcast:**

```
int MPI_Bcast(void *m, int count, MPI_Datatype dtype, int root, MPI_Comm comm)
```

- Envía un mensaje desde un proceso (**root**) al resto de los procesos del comunicador (**comm**).



- **MPI_Scatter:** Un proceso (**root**) distribuye un conjunto de datos, proporcionalmente y según rank, entre los procesos (incluyéndose a si mismo) de un comunicador.



```
int MPI_Scatter(void* sendM,  
               int sendcount,  
               MPI_Datatype sendtype,  
               void* recvM,  
               int recvcount,  
               MPI_Datatype recvtype,  
               int root,  
               MPI_Comm comm)
```

sendM: mensaje a enviar.

sendcount: cantidad de elementos enviados a cada proceso.

sendtype: tipo de datos de los elementos del mensaje a enviar.

recvM: puntero a la porción del mensaje a recibir.

recvcount: cantidad de elementos a recibir por cada proceso.

recvtype: tipo de datos de los elementos del mensaje a recibir.

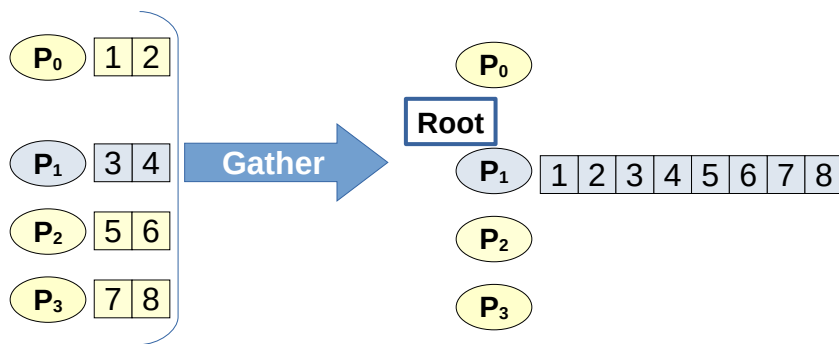
root: rank del proceso que envía el mensaje.

comm: comunicador.

*sólo tienen sentido para **root***



- **MPI_Gather:** inversa de MPI_Scatter. Cada proceso (incluyendo a **root**) envía datos al proceso **root** que organiza los datos recibidos según el rank de los procesos.



```
int MPI_Gather(void* sendM,  
              int sendcount,  
              MPI_Datatype sendtype,  
              void* recvM,  
              int recvcount,  
              MPI_Datatype recvtype,  
              int root,  
              MPI_Comm comm)
```

sendM: mensaje a enviar.

sendcount: cantidad de elementos enviados por cada proceso.

sendtype: tipo de datos de los elementos del mensaje a enviar.

recvM: puntero al mensaje a recibir.

recvcount: cantidad de elementos a recibir por cada proceso.

recvtype: tipo de datos de los elementos del mensaje a recibir.

root: rank del proceso que recibe el mensaje.

comm: comunicador.

*sólo tienen sentido para **root***



```
#include<mpi.h>
#define N ...

int main( int argc, char *argv[]){
    int nProcs;
    int id;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id);
    MPI_Comm_size(MPI_COMM_WORLD,&nProcs);

    if (id == 0) {
        root(nProcs);
    } else {
        worker(nProcs);
    }

    MPI_Finalize();
    return 0;
}
```

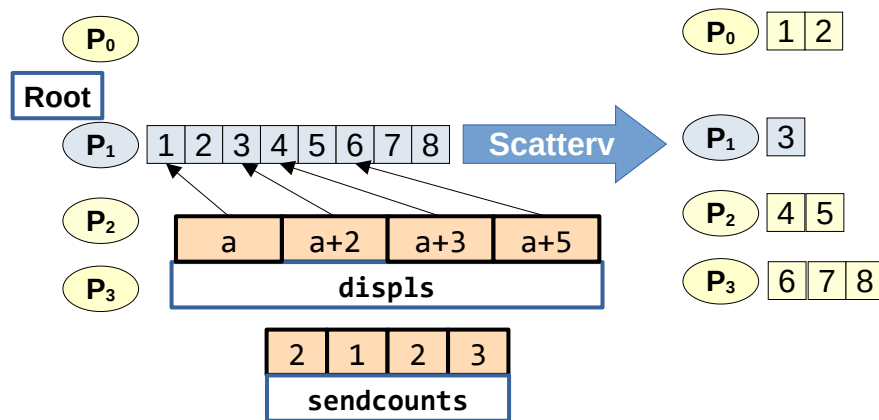
```
void root(int nProcs){
    int m[N];
    nPart = N/nProcs;
    int p[nPart];

    ...
    MPI_Scatter(m,nPart,MPI_INT,p,nPart,MPI_INT,0,MPI_COMM_WORLD);
    ... // Procesa su parte
    MPI_Gather(p,nPart,MPI_INT,m,nPart,MPI_INT,0,MPI_COMM_WORLD);
}

void worker(int nProcs){
    int *m = NULL;
    nPart = N/nProcs;
    int p[nPart];

    ...
    MPI_Scatter(m,nPart,MPI_INT,p,nPart,MPI_INT,0,MPI_COMM_WORLD);
    ... // Procesa su parte
    MPI_Gather(p,nPart,MPI_INT,m,nPart,MPI_INT,0,MPI_COMM_WORLD);
}
```

- **MPI_Scatterv**: distribuye bloques de datos de tamaño variable.



```
int MPI_Scatterv(const void* sendM,
                 const int* sendcounts,
                 const int* displs,
                 MPI_Datatype sendtype,
                 void* recvM,
                 int recvcount,
                 MPI_Datatype recvtype,
                 int root,
                 MPI_Comm comm)
```

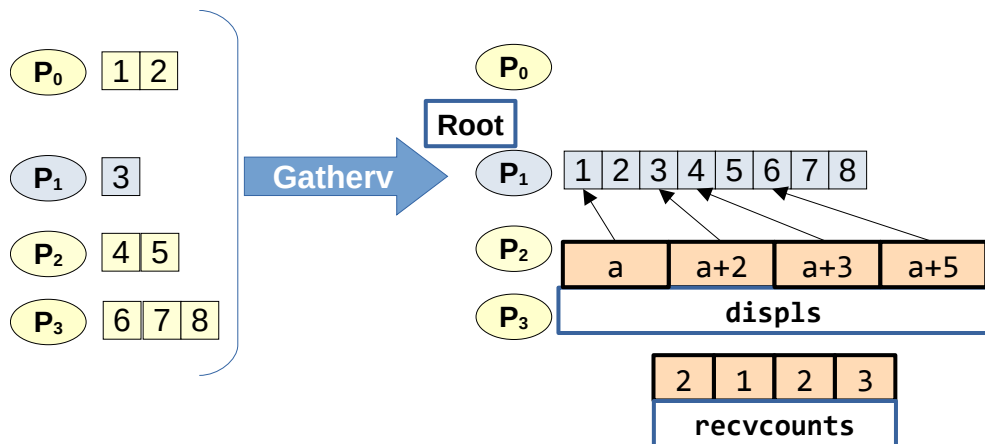
Parámetros adicionales:

displs: desplazamiento de cada bloque dentro del arreglo de envío.

sendcounts: arreglo de enteros que indica cuántos elementos recibe cada proceso.

Al proceso **rank=i** le corresponderán **sendcounts[i]** datos, comenzando desde la posición **displs[i]** relativa al arreglo **a**.

- **MPI_Gatherv**: recolectar bloques de datos de tamaño variable.



```
int MPI_Gatherv(const void* sendM,
                const int sendcounts,
                MPI_Datatype sendtype,
                void* recvM,
                int* recvcounts,
                const int* displs,
                MPI_Datatype recvtype,
                int root,
                MPI_Comm comm)
```

Parámetros adicionales:

displs: desplazamiento de cada bloque dentro del arreglo de recepción.

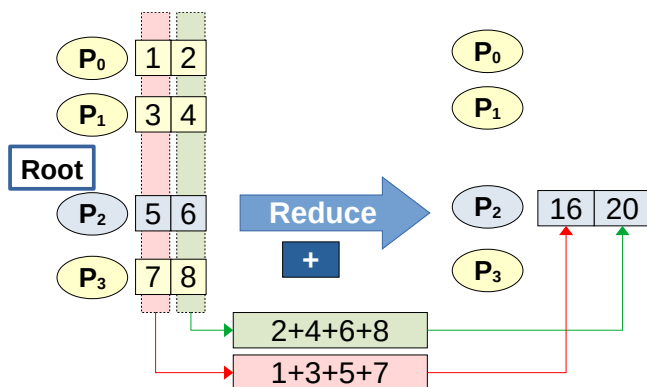
recvcounts: arreglo de enteros que indica cuántos elementos recibe **root** de cada proceso.

*sólo tienen sentido para **root***

root recibirá de un proceso **rank=i** **recvcounts[i]** datos, comenzando desde la posición **displs[i]** relativa al arreglo **a**.



- **MPI_Reduce:** cada proceso posee uno o varios valores y deben combinarse en un resultado global que quedará en un proceso (**root**)



```
int MPI_Reduce(const void* sendM,  
               void* recvM,  
               int count,  
               MPI_Datatype dtype,  
               MPI_Op op,  
               int root,  
               MPI_Comm comm)
```

sendM: puntero con valores a enviar.

recvM: puntero a los resultados globales.

count: cantidad de elementos a reducir.

dtype: tipo de datos de los elementos.

op: operación a aplicar.

root: rank del proceso que recibe los resultados.

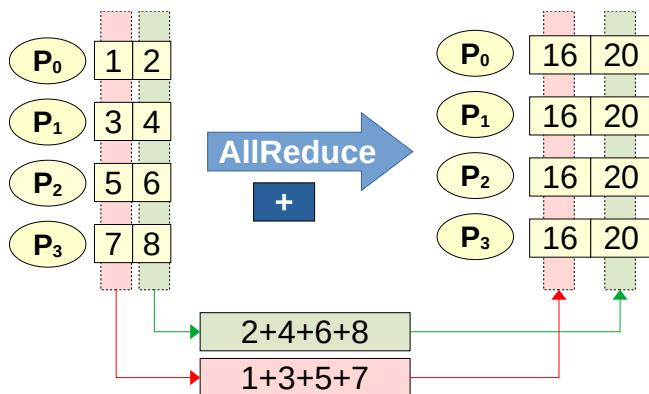
comm: comunicador.

*sólo tiene sentido para **root***



- **MPI_MAX**: Máximo entre los elementos
- **MPI_MIN**: Mínimo entre los elementos
- **MPI_SUM**: Suma
- **MPI_PROD**: Producto
- **MPI_LAND**: AND lógico (devuelve 1 o 0, verdadero o falso)
- **MPI_BAND**: AND a nivel de bits
- **MPI_LOR**: OR lógico
- **MPI_BOR**: OR a nivel de bits
- **MPI_LXOR**: XOR lógico
- **MPI_BXOR**: XOR a nivel de bits
- **MPI_MAXLOC**: Valor máximo entre los elementos y el rango del proceso que lo tenía
- **MPI_MINLOC**: Valor mínimo entre los elementos y el rango del proceso que lo tenía

- **MPI_Allreduce:** los valores finales quedan distribuidos en todos los procesos (combinación de reducción + broadcast).



```
int MPI_Reduce(const void* sendM,  
               void* recvM,  
               int count,  
               MPI_Datatype dtype,  
               MPI_Op op,  
               MPI_Comm comm)
```

Prototipo Ídem MPI_Reduce
Se omite el parámetro root

MPI

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales obre MPI

- **Afinidad** (*process binding*): depende de cada distribución.
 - MPI no define un estándar dentro de la API para afinidad.
 - Generalmente, se realiza al ejecutar el programa desde la línea de comandos.
 - Cada distribución define cómo interpreta la lista de nodos y decide qué proceso se ejecuta en qué unidad de procesamiento.
 - Dentro del programa, depende del sistema operativo (`sched_setaffinity`), mas complejo.

MPI

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales obre MPI

- Procesos pueden quedar inactivos esperando recibir o enviar un mensaje mediante operaciones bloqueantes, sin realizar trabajo útil.

¿Puede mitigarse esa espera?



- Operaciones colectivas son muy útiles pero:



- A veces pueden generar esperas innecesarias.
- Si los nodos no poseen capacidad necesaria no podemos usarlas con grandes volúmenes de datos y debemos usar operaciones punto a punto.

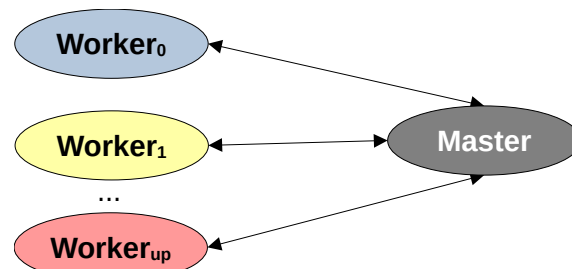


- Operaciones punto a punto no bloqueantes (Isend, Irecv) para aplicar dos técnicas:
 - **Ocultamiento de la latencia** (*Latency hiding*)
 - O **superposición** (*Overlapping*) cómputo y comunicación.
 - **Carga anticipada** (*prefetching*): se solicitan datos para el próximo cálculo.

Ocultamiento de la latencia - Ejemplo

51

- Mapeo dinámico centralizado *Master-Worker*:
 - Workers piden trabajo al Master.
 - Master demora en preparar el trabajo para cada Worker.
 - Workers reciben trabajo hasta que el Master indique que no hay trabajo disponible.



Process Master

```
int workerId;  
  
while ("hay trabajo disponible")  
    "Prepara trabajo para enviar"  
    Receive(pedidoTrabajo,workerId)  
    Send(trabajo,workerId)  
end while  
  
for worker = 1 a numWorkers  
    Receive(pedidoTrabajo,workerId)  
    Send("No mas Trabajo",workerId)  
end for  
End Master
```



Process Worker[workerId:1..numWorkers]

```
Send(pedidoTrabajo,Master)  
Receive(trabajo,Master)  
while ("haya" trabajo)  
    "Trabaja"  
    Send(pedidoTrabajo,master)  
    Receive(trabajo,master)  
end while  
End worker
```

Ocultamiento de la latencia - Ejemplo

52

- Aplicamos ocultamiento de latencia y carga anticipada:



- Master no cambia 🤔

- Workers piden trabajo al Master, lo reciben y piden trabajo para la próxima iteración mediante operaciones no bloqueantes.



Ocultamiento y carga anticipada

Process Master
int workerId;

```
while ("hay trabajo disponible")
  "Prepara trabajo para enviar"
  Receive(pedidoTrabajo, workerId)
  Send(trabajo, workerId)
end while
```

```
for worker = 1 a numWorkers
  Receive(pedidoTrabajo, workerId)
  Send("No mas Trabajo", workerId)
end for
```

End Master

Mientras un worker trabaja va recibiendo trabajo en segundo plano (Ireceive) para la próxima iteración.

Espera que Ireceive se complete.

Intercambio de punteros a los buffers.

Process Worker[workerId:1..numWorkers]

```
Send(pedidoTrabajo1, Master)
Receive(trabajo1, Master)
while ("haya" trabajo1)
  ISend(pedidoTrabajo2, Master)
  IReceive(trabajo2, Master)
  "Trabaja"
  Wait(IReceive)
  swap(trabajo1, trabajo2)
end while
```

End worker

MPI

Funcionamiento, compilación y ejecución

Estructura de programa

Comunicación:

Parámetros comunes de las funciones MPI

Comunicación punto a punto

Comunicación Colectiva

Afinidad

Ocultamiento de la latencia

Comentarios finales obre MPI

- MPI nace para facilitar la programación paralela en clusters moncore (siglo XX).
- Podemos implementar aplicaciones MPI en arquitecturas de memoria compartida:
 - Desventaja: mayor consumo de memoria
 - Espacio para almacenar mensajes.
 - Replicación de datos.
 - Mayor *overhead* al enviar y recibir datos que acceder directamente a memoria.
- MPI está en constante evolución, incorporando nueva funcionalidad en cada versión.

